Building femm-qt on Linux

D. Meeker

August 23, 2026

1 Overview

femm-qt is a cross-platform port of FEMM (Finite Element Method Magnetics) using Qt 6 for the GUI and CMake/Ninja for the build system. This document describes how to build it from source on Linux. The instructions target Ubuntu 24.04 LTS but should work with minor changes on other distributions (Debian, Fedora, Arch, etc.).

The build produces six executables:

- `femm_gui` — the main GUI application
- `femm_headless` — headless IPC server (no GUI)
- `fkn` — magnetics solver
- `belasolv` — electrostatics solver
- `hsolv` — heat flow solver
- `csolv` — current flow solver

2 Prerequisites

2.1 Build Tools

Install the compiler toolchain, CMake, Ninja, and git:

```
sudo apt update
sudo apt install build-essential cmake ninja-build git
```

This provides `g++`, `cmake`, `ninja`, and `git`.

2.2 Qt 6

Install the Qt 6 development libraries:

```
sudo apt install qt6-base-dev libqt6widgets6
```

On Ubuntu 24.04, this installs Qt 6.4 or later from the standard repositories. No separate Qt installer or online account is needed.

To verify the installation:

```
qmake6 --version
```

2.3 Other Distributions

On Fedora:

```
sudo dnf install gcc-c++ cmake ninja-build git qt6-qtbase-devel
```

On Arch Linux:

```
sudo pacman -S base-devel cmake ninja git qt6-base
```

3 Getting the Source

If you have a source zip, unzip it:

```
unzip femm-qt-src.zip
cd femm-qt
```

Or, if the source is hosted on GitHub, clone the repository:

```
git clone <repository-url> femm-qt
cd femm-qt
```

4 Building

From the `femm-qt` directory, run the build script:

```
./linux_build.sh
```

This checks that the build tools are present, configures the project with CMake, and builds it with Ninja. All executables are placed in `build/bin/`.

To wipe the build directory and rebuild from scratch, pass `clean`:

```
./linux_build.sh clean
```

If the build fails because CMake cannot find Qt 6, see *Manual configuration* under Troubleshooting.

5 Installing

You can run femm-qt from `build/bin` indefinitely; nothing requires an install. To put it somewhere permanent instead:

```
./linux_build.sh install
```

This installs what is already built — it does not rebuild — into your home directory:

```
~/.local/opt/femm-qt/   the program, its solvers, the material
                        libraries, the bindings, and a pristine
                        set of the examples
~/.local/bin/           femm_gui, femm_headless, the four solvers
                        (fkn, belasolv, hsolv, csolv), lua4, and
                        femm-lua
~/.local/share/         menu entry and icon
```

```
~/Documents/FEMM/
       examples/        your copy of the examples
       docs/            the manual and the scripting references
```

The solvers are linked as commands of their own because each is usable that way: `fkn model` solves `model.fem` and writes `model.ans`, with `belasolv`, `hsolv` and `csolv` doing the same for electrostatics, heat flow and current flow — which is what you want from a Makefile or a batch queue. `lua4` is a stock Lua 4 interpreter and runs directly too.

The examples are installed twice on purpose. The program directory is out of the way — and on Ubuntu worse than that, since the file manager hides `~/.local` outright — so a second copy goes somewhere you can actually open, under whatever your desktop calls *Documents*. The shipped examples in it are replaced on every install, so that a corrected example reaches you rather than only new users — and because *running* an example rewrites its `.fem` (analysis saves the document first), which makes any "has this been edited?" test useless here. Anything else you keep in that directory is untouched: only the files femm-qt ships are written. Work in a copy, which is what every example script does on its second line.

FEMM then appears in the application menu. For `femm_gui` to work as a command, `~/.local/bin` has to be on your `PATH`, and **the installer does not put it there without being asked**. Your shell startup files are your configuration — kept in a dotfiles repository, synced between machines, or managed for you on an institutional login — and an installer editing one uninvited produces a confusing diff at best and a change that is silently reverted at worst.

So it checks, and tells you. If the directory is already on `PATH` it says nothing at all. On Debian and Ubuntu the stock `~/.profile` already adds `~/.local/bin` when it exists, so it will be there at your next login and the installer leaves well alone. Otherwise it prints the one line to add and which file to add it to.

If you would rather it did the edit, ask for it:

```
./linux_build.sh install --modify-path
```

which appends a single marked block, once — running it again finds the block and adds nothing.

Which file that is depends on your shell, and so does when the change takes effect. `~/.profile` is read at login by the desktop session, and every terminal inherits the environment from there, so a bash or `sh` user must log out and back in — opening a new terminal window is *not* enough. `~/.zshrc` and fish's `config.fish` are read by every interactive shell, so there a new terminal really is sufficient. Under `csh` or `tcsh` nothing is written at all: those read none of these files and use a different syntax entirely, so the installer prints the `set path` line for `~/.cshrc` rather than leave a line that would never run.

Whatever you do, it cannot fix the shell you ran it from — no program can change the environment of the shell that launched it.

Pass a different prefix if you would rather it went elsewhere:

```
./linux_build.sh install /opt/femm
```

It installs into your home rather than `/usr/local` on purpose. FEMM finds its solvers, scripting assets and material libraries beside its own executable, so a single self-contained directory needs no special handling — and, more importantly, the material libraries and the examples stay *writable*. Both need to be: the material editor saves the libraries, and every example script opens a model and then immediately saves it under another name. In a system directory they would be read-only and neither would work.

Uninstalling

```
./linux_build.sh uninstall
```

It removes the program directory, the commands, the menu entry, the icon and the documents, and takes out the `PATH` line if it was the one that added it. Pass the same prefix you installed with if it was not the default, and `-y` to skip the confirmation.

It works from a manifest the install leaves in the program directory rather than from a list of paths worked out afresh, because several of those paths are not reliably re-derivable later — which solver commands were linked, whether a profile was edited, and above all where the documents went, since that comes from `xdg-user-dir` and can be reconfigured. A program directory with no manifest is refused rather than guessed at, and a command in `~/.local/bin` that does not point into the program directory is left alone: a name like `lua4` could easily belong to something else.

Your copy of the examples is never removed. Those are yours, an install never overwrites them, and a script that deletes a directory somebody has been working in is not one to run casually.

6 Running

Launch the GUI:

```
./bin/femm_gui
```

Or the headless server:

```
./bin/femm_headless
```

If you installed as above, use the copies on your `PATH` instead, from any directory:

```
femm_gui
femm_headless
```

6.1 Setting FEMM_DIR

The language bindings (OctaveFEMM, MathFEMM, pyFEMM, JuliaFEMM) locate the FEMM executables via the `FEMM_DIR` environment variable. **If you installed, you can skip this.** Each binding looks for `femm_gui` and `femm_headless` in three places, in order: `FEMM_DIR`, then beside its own directory, then *every directory on `PATH`* — and the install puts them on `PATH`. So the bindings work with nothing set at all.

`FEMM_DIR` remains the way to override that, which is what you want when running from a build tree that was never installed, or when you have more than one build and need to say which:

```
export FEMM_DIR=/path/to/femm-qt/build/bin
```

Add it to your login profile to make it persistent. Being first in the list, it beats both the relative search and `PATH`.

The binding files themselves are in `bindings/` under the program directory — `mfiles` for OctaveFEMM, and `mathfemm`, `scifemm`, `juliafemm`, `pyfemm` and `luafemm` beside it.

6.2 Desktop Integration (Optional)

The `install` step above does this for you, with an icon. The recipe below is for running in place out of `build/bin`.

To add femm-qt to your application menu, create a `.desktop` file:

```
cat > ~/.local/share/applications/femm-qt.desktop << 'EOF'
[Desktop Entry]
Name=FEMM
Comment=Finite Element Method Magnetics
Exec=/path/to/femm-qt/build/bin/femm_gui
Type=Application
Categories=Science;Engineering;
EOF
```

7 Using with MATLAB or Octave

7.1 Octave

Install Octave and its Java support:

```
sudo apt install octave default-jdk
```

Note that in Ubuntu, the version of Octave installed by App Center does *not* have Java support, so the installation by `apt`, which does have Java support, is needed. Find the path to Java by typing:

```
readlink -f $(which java)
```

at the Bash command line. This usually returns something like:

```
/usr/lib/jvm/java-21-openjdk-amd64/bin/java
```

Edit your `~/.bashrc` file and add as the last line:

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
```

(`FEMM_DIR` used to be needed here as well. It no longer is: `openfemm` searches `PATH` for `femm_gui` and `femm_headless`, and an install puts them there. Set it only to choose between several builds.) Run Octave and confirm that Java is running by typing

```
version -java
```

at the Octave prompt. It should return something like:

```
ans = Java 21.0.10+7-Ubuntu-124.04 with Ubuntu OpenJDK 64-Bit Server VM mixed mode, sharing
```

which indicates that Java is properly installed. Then add the path to the the OctaveFEMM mfiles to Octave by typing the following at the Octave command line:

```
addpath("~/.local/opt/femm-qt/bindings/mfiles")
savepath
```

or, if you did not install and are working from the source tree, `addpath("/path/to/femm-qt/octavefemm")`. `savepath` is what makes it stick; `~/.octaverc` does the same job.

This is a *different* search path from the one above, and the two are easy to confuse. Octave's path is how Octave finds `openfemm.m`; the system `PATH` is how `openfemm.m` then finds `femm_gui`. Only the second is automatic. The install prints the `addpath` line for every language when it finishes.

There is a fuller OctaveFEMM guide in `~/Documents/FEMM/docs/octavefemm.md`, installed alongside the manual.

7.2 MATLAB

MATLAB on Linux uses its bundled JRE for socket communication with femm-qt, which should work without additional configuration.

As with Octave, the executables are found on `PATH` and need no setting up; what MATLAB needs is the OctaveFEMM mfiles on *its* path, through `pathtool` or `addpath('~/.local/opt/femm-qt/bindings/mfiles`. If you keep more than one build and need to say which, set `FEMM_DIR` — before launching MATLAB, or from inside it:

```
setenv('FEMM_DIR', '/path/to/femm-qt/build/bin')
```

7.3 Python

The Python interface is/will be available as pyfemm-qt. Install it via:

```
pip install pyfemm-qt
```

To install the version that came with this source instead, change to the `pyfemm` directory — in the source tree, or `~/.local/opt/femm-qt/bindings/pyfemm` after an install — and type:

```
pip install -e .
```

`-e` records the *absolute path* of that directory, so an editable install breaks if the directory is later moved or deleted. Point it at the installed copy, not at a build tree you might clean.

Without installing at all, `sys.path.append` on the same directory works just as well, since pyfemm is a single file with no dependencies. On Ubuntu 24.04 and later, pip may refuse to install into the system Python with an "externally-managed-environment" error. Since pyfemm is a single file with no dependencies, it is safe to override this:

```
pip install -e . --break-system-packages
```

A fuller pyFEMM guide — multi-instance use, headless mode, connecting to an instance that is already running — is installed at `~/Documents/FEMM/docs/pyfemm.md`.

8 Troubleshooting

8.1 Manual configuration

The build script just wraps the standard CMake and Ninja steps. To run them by hand:

```
mkdir build && cd build
cmake .. -G Ninja -DCMAKE_BUILD_TYPE=Release
ninja
```

CMake finds Qt 6 from the system packages automatically. If it does not, pass the Qt prefix explicitly:

```
cmake .. -G Ninja \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_PREFIX_PATH=/usr/lib/x86_64-linux-gnu/cmake/Qt6
```

8.2 Qt not found

If CMake reports that it cannot find Qt6, check that the development package is installed:

```
dpkg -l | grep qt6-base-dev
```

If installed but not found, pass `-DCMAKE_PREFIX_PATH` pointing to the Qt 6 CMake config directory, typically `/usr/lib/x86_64-linux-gnu/cmake/Qt6`.

8.3 Missing OpenGL headers

On minimal server installations or VMs without a desktop environment, Qt may require OpenGL development headers:

```
sudo apt install libgl1-mesa-dev libglu1-mesa-dev
```

8.4 VirtualBox Guest Additions

If running in a VirtualBox VM, install Guest Additions for proper display scaling and shared folders:

```
sudo apt install virtualbox-guest-utils \
    virtualbox-guest-x11
```

Reboot after installation. For shared folders, add your user to the `vboxsf` group:

```
sudo usermod -aG vboxsf $USER
```

8.5 MESA / OpenGL errors in VirtualBox

VirtualBox's virtual GPU may not support the OpenGL version Qt expects, producing errors like "ZINK: failed to choose pdev" or "failed to create dri2 screen." Two fixes:

1. Enable 3D Acceleration in VirtualBox (Settings → Display → check "Enable 3D Acceleration" and move the slider to at least 128MB video memory out in Expert mode), or
2. Force Qt to use software rendering by adding

   ```
   export LIBGL_ALWAYS_SOFTWARE=1
   ```

   to `~/.bashrc`